
Enhancing CondorBrain (Mamba-2) for Institutional-Grade SPY Options Trading

A High-Fidelity Research Report with Restored Math + Embedded Implementation Snippets

Targets: NP/DD, Sharpe, stability, sub-second 1-minute inference on A100/H100 (*i.e. tick data*)

0. Abstract

CondorBrain is already a strong foundation:

1. Deep Mamba-2 (Selective-Scan SSM)
2. 24-feature 1-minute input
3. Multi-head outputs
4. Fused AdamW,
5. BF16/TF32 enabled
6. Massive, aligned dataset

1. Exact data expectations

- Explicit CSV schemas
- Timestamp and causality requirements
- Spot vs options separation
- Lag treated as a first-class reality (not swept under the rug)

This prevents the classic “*we couldn’t reproduce it because the data wasn’t specified*” critique.

2. Minimal run path

A clean, auditable, **5-step pipeline**:

1. Load data
2. Stream `MarketSnapshot`
3. Neural inference
4. Neural-fuzzy fusion
5. Sizing + trade intent

No brokers. No magic. No hidden steps.

This makes CondorIntelligence™ usable as a **drop-in evaluation / benchmark test harness**.

The identified bottleneck is **training VRAM** at high batch + deep model, and the upside is **risk-aligned learning** (directly training what is scored: NP/DD, Sharpe, DD, turnover), plus **regime specialization** without latency blow-ups. A first of its kind on such a massive learning dataset to construct a Neural Net model with so many capabilities and forward predictors fully integrated and baked into one model file for rapid determination.

This report, which I have written to tighten down the nuts and bolts and add to its already Nnovel approach of advanced features, gives:

1. **A100/H100 CUDA speed stack** (allocator, TF32/BF16, compile, graphs) – **Done and incorporated in the most recent modelling run.**
2. **OOM fixes** that preserve **effective batch = 256**
3. **Architecture upgrades** that keep **O(N)** scaling while adding select non-SSM capacity
4. **Risk-aligned objectives** that directly optimize NP/DD & drawdown behavior
5. **Order-book (L2) + advanced indicators** for institutional entry/exit prediction
6. **Evaluation protocol** for non-leaky, realistic PnL and robustness

1. High-Performance CUDA / Training Stack (A100/H100)

1.1 Allocator & fragmentation control (critical for “slightly over” OOM)

Use these environment variables before launching training:

```
export
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True,max_split_size_mb:512,garbage_collection_threshold:0.9
export CUDA_DEVICE_MAX_CONNECTIONS=1
export OMP_NUM_THREADS=8
export MKL_NUM_THREADS=8
```

1.2 Tensor Core math (TF32) + cuDNN autotune

```
1. import torch
2. torch.backends.cuda.matmul.allow_tf32 = True
3. torch.backends.cudnn.allow_tf32 = True
4. torch.backends.cudnn.benchmark = True
5. torch.set_float32_matmul_precision("high")
6.
```

1.3 BF16 autocast (preferred on A100/H100)

- BF16 preserves FP32-like dynamic range while using tensor cores.

```
1. import torch
2. from torch.cuda.amp import autocast, GradScaler
3.
4. USE_BF16 = torch.cuda.is_available() and torch.cuda.get_device_capability(0)[0] >= 8
5. amp_dtype = torch.bfloat16 if USE_BF16 else torch.float16
6. scaler = GradScaler(enabled=not USE_BF16)
7.
```

1.4 torch.compile for kernel fusion and Python overhead reduction

```
1. model = torch.compile(model, mode="max-autotune", fullgraph=False)
2.
```

1.5 CUDA graphs for sub-second inference in production

```
1. import torch
2.
3. class CUDAGraphRunner:
4.     def __init__(self, model, example_batch):
5.         self.model = model.eval()
6.         self.static_x = example_batch.clone()
7.         self.graph = torch.cuda.CUDAGraph()
8.
9.         # Warmup
10.        for _ in range(3):
11.            _ = self.model(self.static_x)
12.        torch.cuda.synchronize()
13.
14.        with torch.cuda.graph(self.graph):
15.            self.static_y = self.model(self.static_x)
16.
17.        @torch.no_grad()
18.        def __call__(self, x):
19.            self.static_x.copy_(x)
20.            self.graph.replay()
21.            return self.static_y
22.
```

2. Fixing OOM While Keeping Effective Batch = 256

Your crash: CUDA OOM by ~128 MiB on an 80 GB class GPU implies **activation peak + fragmentation**.

2.1 Keep “effective batch 256” via microbatch + accumulation

Let microbatch = 64 and grad_accum = 4:

```
1. grad_accum = 4
2. optimizer.zero_grad(set_to_none=True)
3.
4. for step, (x, y) in enumerate(loader):
5.     x = x.cuda(non_blocking=True)
6.     y = y.cuda(non_blocking=True)
7.
8.     with autocast(dtype=amp_dtype):
9.         out = model(x, return_regime=True, forecast_days=0)
10.        loss = compute_loss(out, y) / grad_accum
11.
12.    if scaler.is_enabled():
13.        scaler.scale(loss).backward()
14.    else:
15.        loss.backward()
16.
17.    if (step + 1) % grad_accum == 0:
18.        if scaler.is_enabled():
19.            scaler.step(optimizer)
20.            scaler.update()
21.        else:
22.            optimizer.step()
23.        optimizer.zero_grad(set_to_none=True)
24.
```

2.2 Activation checkpointing across Mamba blocks (big VRAM win)

Checkpointing trades compute for memory—perfect for “slightly over” OOM.

```
1. import torch.utils.checkpoint as ckpt
2.
3. def checkpointed_forward(layers, x):
4.     for layer in layers:
5.         x = ckpt.checkpoint(layer, x, use_reentrant=False)
6.     return x
7.
```

In the current model forward, replace:

```
1. for layer in self.layers:
2.     x = layer(x)
3.
```

with:

```
1. x = checkpointed_forward(self.layers, x)
2.
```

2.3 Reduce peak allocation spikes (small but decisive)

```
1. optimizer.zero_grad(set_to_none=True)
2. torch.cuda.empty_cache() # use sparingly; helps after large transients
3.
```

3. Mathematical Core: CondorBrain with Mamba-2 SSM

3.1 Input feature vector

$$\mathbf{x}_t \in \mathbb{R}^{24}$$

Latent embedding:

$$\mathbf{E}_t = \text{LayerNorm}(\mathbf{W}_{\text{in}} \mathbf{x}_t + \mathbf{b}_{\text{in}})$$

Where:

- $\mathbf{W}_{\text{in}} \in \mathbb{R}^{D \times 24}$, $\mathbf{E}_t \in \mathbb{R}^D$, $D = d_{\text{model}}$

3.2 Mamba layer recurrence (conceptual)

Continuous-time SSM:

$$\begin{aligned} \mathbf{h}'(t) &= \mathbf{A} \mathbf{h}(t) + \mathbf{B} \mathbf{x}(t) \\ \mathbf{y}(t) &= \mathbf{C} \mathbf{h}(t) \end{aligned}$$

Discretized form:

$$\begin{aligned} \mathbf{h}_t &= \bar{\mathbf{A}}_t \mathbf{h}_{t-1} + \bar{\mathbf{B}}_t \mathbf{x}_t \\ \mathbf{y}_t &= \mathbf{C} \mathbf{h}_t \end{aligned}$$

Selective scan means $\bar{\mathbf{A}}_t$ (and sometimes $\bar{\mathbf{B}}_t$) depend on the input context and learned Δ step sizes, enabling dynamic “remember/forget”.

3.3 Gated output mixing per layer

$$\begin{aligned} \mathbf{h}_t^{(\ell)} &= \text{SSM}(\mathbf{A}, \mathbf{B}, \mathbf{C}, \Delta, \mathbf{E}_t^{(\ell-1)}) \\ \mathbf{E}_t^{(\ell)} &= \text{SiLU}(\mathbf{W}_{\text{gate}} \mathbf{E}_t^{(\ell-1)}) \odot \mathbf{h}_t^{(\ell)} \end{aligned}$$

4. Architecture Enhancements That Preserve $O(N)$ and Low Latency

4.1 Volatility-Gated Sparse Attention (insert rarely)

Goal: add limited global mixing only when needed (e.g., regime shift).

Gate:

$$g_t = \sigma(u^T z_t + b) \in (0,1)$$

Hybrid mix:

$$E_t^+ = (1 - g_t)E_t + g_t \text{Attn}(E_{t-w:t})$$

```
1. import torch, torch.nn as nn
2.
3. class VolGatedAttn(nn.Module):
4.     def __init__(self, d_model, n_heads=8):
5.         super().__init__()
6.         self.attn = nn.MultiheadAttention(d_model, n_heads, batch_first=True)
7.         self.gate = nn.Sequential(
8.             nn.Linear(d_model, d_model//4),
9.             nn.SiLU(),
10.            nn.Linear(d_model//4, 1),
11.            nn.Sigmoid()
12.        )
13.
14.     def forward(self, E):
15.         g = self.gate(E[:, -1, :]).unsqueeze(1) # (B,1,1)
16.         A, _ = self.attn(E, E, E, need_weights=False)
17.         return (1.0 - g) * E + g * A
18.
```

Where: after layer 8, 16, 24 (3 inserts max).

4.2 Regime-Routed MoE (Top-K experts)

Router logits:

$$r_t = \text{softmax}(G(E_t))$$

Top-K mixture:

$$Y_t = \sum_{e \in \text{TopK}(r_t)} r_{t,e} F_e(E_t)$$

```
1. import torch, torch.nn as nn
2.
3. class TopKMoE(nn.Module):
4.     def __init__(self, d_model, n_experts=4, k=1):
5.         super().__init__()
6.         self.k = k
7.         self.gate = nn.Linear(d_model, n_experts)
8.         self.experts = nn.ModuleList([
9.             nn.Sequential(
10.                 nn.Linear(d_model, 4*d_model),
11.                 nn.SiLU(),
12.                 nn.Linear(4*d_model, d_model),
13.             ) for _ in range(n_experts)
14.         ])
15.
16.     def forward(self, E):
17.         logits = self.gate(E[:, -1, :]) # (B,M)
18.         probs = torch.softmax(logits, dim=-1) # (B,M)
19.         topv, topi = torch.topk(probs, self.k, dim=-1) # (B,k)
20.
21.         out = torch.zeros_like(E)
22.         for j in range(self.k):
23.             idx = topi[:, j]
24.             w = topv[:, j].view(-1, 1, 1)
25.             expert_out = torch.stack([self.experts[idx[b]](E[b]) for b in range(E.size(0))],
26.                                     dim=0)
27.             out = out + w * expert_out
28.         return out
```

Latency note: Use k=1 and only 1–2 MoE inserts.

4.3 Differentiable Neuro-Fuzzy Safety Overlay (policy shaping)

Membership vector $\mu \in [0,1]^R$

Rule correction:

$$\Delta a = (\mu W_{\text{rule}}) / (\sum \mu + \epsilon)$$

Final action:

$$a' = a + \Delta a$$

```
1. import torch, torch.nn as nn
2.
3. class NeuroFuzzyOverlay(nn.Module):
4.     def __init__(self, action_dim=8, n_rules=16):
5.         super().__init__()
6.         self.mu_net = nn.Sequential(
7.             nn.Linear(action_dim, 64),
8.             nn.SiLU(),
9.             nn.Linear(64, n_rules),
10.             nn.Sigmoid()
11.         )
```

```
12.         self.rule_delta = nn.Parameter(torch.zeros(n_rules, action_dim))
13.         self.eps = 1e-6
14.
15.     def forward(self, a):
16.         mu = self.mu_net(a)                # (B,R)
17.         num = mu @ self.rule_delta         # (B,A)
18.         den = mu.sum(dim=-1, keepdim=True) + self.eps
19.         return a + num / den
20.
```

5. “Never Been Done Before” (*but mathematically coherent*) Enhancements

These are creative but grounded and implementable.

5.1 Distributionally-Robust Drawdown Minimization (DRO-DD)

Instead of minimizing average loss, minimize worst-case tail loss across regimes.

For batch losses ℓ_i , define robust objective:

$$L_{\text{DRO}} = \tau \log(\sum \exp(\ell_i / \tau))$$

This forces training to “care” about the hardest, drawdown-heavy scenarios.

```
1. import torch
2.
3. def dro_logsumexp(losses, tau=0.05):
4.     return tau * torch.logsumexp(losses / tau, dim=0)
5.
```

Use ℓ_i = per-sample DD penalty or negative NP/DD proxy.

5.2 “Gamma-Shock Adversarial Training” (options-aware perturbations)

Create adversarial perturbations to features that mimic real risk events:

- IV jump
- skew steepening
- spread widening
- gap move

Let x be input, create $x_{\text{adv}} = x + \delta$ where δ maximizes risk loss subject to constraint $\|\delta\| \leq \epsilon$.

Maximization step:

$$\delta^* = \operatorname{argmax}_{\{\|\delta\| \leq \epsilon\}} L_{\text{risk}}(\text{model}(x + \delta))$$

Then train on both x and x_{adv} to harden model against tail regimes.

```
import torch
```

```
1. def fgsm_adversary(x, loss_fn, eps=1e-2):
2.     x = x.detach().requires_grad_(True)
3.     loss = loss_fn(x)
4.     loss.backward()
5.     return (x + eps * x.grad.sign()).detach()
6.
```

5.3 Regime-Conditional Weight Averaging (RC-SWA)

Maintain separate SWA weight averages per regime bucket (low/normal/high vol).
During inference, interpolate weights based on regime probability vector p .

$$\theta = p_{\text{low}} \theta_{\text{low}} + p_{\text{norm}} \theta_{\text{norm}} + p_{\text{high}} \theta_{\text{high}}$$

This acts like a low-latency “weight MoE” without routing activations.

6. Risk-Aligned Training Objectives (Optimize what you score)

You want NP/DD ratio and Sharpe. So the loss must include **drawdown** and **return stability** proxies.

6.1 Soft maximum drawdown penalty

Equity curve $P_t = \sum r_t$

Peak $M_t = \max_{\{u \leq t\}} P_u$

Drawdown $DD_t = M_t - P_t$

Soft max:

$$L_{\text{DD}} = \tau \log(\sum \exp(DD_t / \tau))$$

```
1. import torch
2.
3. def softmax_drawdown(returns, tau=0.02, eps=1e-9):
4.     P = returns.cumsum(dim=-1)
5.     # running max
```

```
6. M = torch.cummax(P, dim=-1).values
7. DD = M - P
8. return tau * torch.log(torch.exp(DD / tau).sum(dim=-1) + eps).mean()
```

6.2 Sharpe proxy

$\mu = \text{mean}(r)$
 $\sigma = \text{std}(r)$

$$L_{\text{Sharpe}} = -\mu / (\sigma + \epsilon)$$

```
1. def sharpe_proxy(returns, eps=1e-9):
2.     mu = returns.mean(dim=-1)
3.     sig = returns.std(dim=-1) + eps
4.     return -(mu / sig).mean()
5.
```

6.3 Turnover penalty (reduce slippage)

If action vector includes position changes Δp_t :

$$L_{\text{turn}} = \lambda \sum |\Delta p_t|$$

```
1. def turnover_penalty(pos, lam=1e-3):
2.     d = (pos[:, 1:] - pos[:, :-1]).abs()
3.     return lam * d.mean()
4.
```

6.4 Composite training objective

$$L_{\text{total}} = \lambda_1 L_{\text{pred}} + \lambda_2 L_{\text{dir}} + \lambda_3 L_{\text{Sharpe}} + \lambda_4 L_{\text{DD}} + \lambda_5 L_{\text{turn}} + \lambda_6 L_{\text{DRO}}$$

7. Advanced Feature Engineering (Institutional-Grade, Non-News)

7.1 Order book imbalance (L2 ladder)

```
1. I_t = (Σ BidDepth - Σ AskDepth) / (Σ BidDepth + Σ AskDepth + ε)
2. def book_imbalance(bid_depth, ask_depth, eps=1e-9):
3.     num = bid_depth.sum(dim=-1) - ask_depth.sum(dim=-1)
4.     den = bid_depth.sum(dim=-1) + ask_depth.sum(dim=-1) + eps
5.     return num / den
6.
```

7.2 Microprice / queue pressure (high predictive value)

Microprice:

$$MP = (\text{Ask} \times \text{BidSize} + \text{Bid} \times \text{AskSize}) / (\text{BidSize} + \text{AskSize})$$

Queue pressure:

$$QP = \text{BidSize} / (\text{BidSize} + \text{AskSize})$$

7.3 Dynamic / adaptive indicators (non-static)

- Adaptive ATR: ATR_t weighted by realized vol regime
 - Volatility of volatility: $VoV_t = \text{std}(IV \text{ changes})$
 - Spread regime: $SR_t = \text{bid-ask} / \text{mid}$
 - Liquidity score: $LS_t = f(\text{volume, spread, depth})$
-

8. Training Pipeline Improvements (10M rows without RAM blowup)

8.1 LazySequenceDataset: O(1) memory views

Use zero-copy windows: sample i returns $X[i:i+L]$ without materializing.

8.2 Streaming normalization (stable BF16)

$$x_scaled = \text{clip}((x - \mu) / (\sigma + \epsilon), -10, 10)$$

9. Sub-Second Inference Deployment Plan (1-Minute Trading)

9.1 Production path

1. Load model $\rightarrow$ BF16 weights
2. Warmup with fixed shape
3. Capture CUDA graph
4. Each minute: update tensor in place $\rightarrow$ replay $\rightarrow$ decode action

9.2 Fixed shapes required

- Keep (B, L, F) constant; pad or roll window.

10. Evaluation Protocol (No Leakage, Realistic PnL)

10.1 Walk-forward split

- Train: 2024-Q1..Q3
- Validate: 2024-Q4
- Test: 2025-Q1
- Roll forward

10.2 Backtest realism

- Use bid/ask mid with slippage model
- Include spread widening in high vol
- Enforce liquidity gates
- Mark-to-market per leg

11. Implementation Checklist (what to change first)

1. Turn on allocator env + BF16 + grad_accum (effective 256) – **Done!**
2. Add activation checkpointing to Mamba blocks
3. Add drawdown + turnover penalties to loss
4. Insert sparse VolGatedAttn (2–3 spots max)
5. Add L2 imbalance features (low latency, high signal)
6. Production: torch.compile + CUDA graphs inference wrapper

External (From current list; formatted without invented metadata)

8. *Stochastic Optimal Control of Iron Condor Portfolios for Profitability* (arXiv).
9. *A Hybrid Approach for S&P 500 Index Volatility Forecasting* (arXiv).
10. *Vanna Volga and Smile-consistent Implied Volatility Surface of ...* (acfr.aut.ac.nz).
11. *Vanna-Volga methods applied to FX derivatives* (arXiv).
12. *Vanna–Volga pricing* (Wikipedia).
13. *Vanna-volga pricing* (EconStor).
14. *Bayesian Optimization for Hyperparameters Tuning in Neural ...* (ResearchGate).
15. *Taking the Human Out of the Loop: A Review of Bayesian Optimization* (cs.ox.ac.uk).

16. *FastAPI performance tuning / guides* (TokenMetrics, Deepnote, Scribd).
 17. *Higher Returns, Lower Risks: Empirical Evidence from US Options ...* (preprints.org).
-

12. Conclusion

This report provides a coherent, implementable pathway to upgrade CondorBrain into an institutional-grade SPY options decision model by:

- increasing expressivity only where needed (**vol-gated hybrid attention**),
- increasing capacity without latency costs (**sparse MoE**),
- adding interpretable safety controls (**neuro-fuzzy fusion**),
- aligning training directly with trading outcomes (**Sharpe/Drawdown/Turnover losses**),
- and ensuring A100/H100 performance (**mixed precision, fusion, CUDA graphs, allocator tuning**).

References / Bibliography

Internal (Papers written by Dr. T. Jerry Mahabub, Ph.D.)

1. **SPYOptionTrader – Quantor-MTFuzz™ Current Software Test Status 122725 9pm gmt-7** (PDF).
2. **SPYOptionTrader – Mathematical & Logical Decision Framework** (PDF).
3. **SPYOptionTrader – Codex Cross Reference of all functions** (PDF).
4. **Structural Integration and Optimization of the Quantor-MTFuzz Decision Engine** (PDF).
5. **Appendix E Diagrams** (PDF).
6. **Loss Function Specification and Training Error Interpretation** (PDF).
7. **Enhancing the Mamba 2 Neural Network for Advanced Trading and ROI Optimization** (PDF).

Practical Training Strategy (Recommended)

1. **Stage A (Supervised Pretrain):**
Train on predictive targets (direction + volatility/width proxies) using:
$$L_{\text{pred}} + L_{\text{dir}}$$
 2. **Stage B (Risk-Aligned Fine-Tuning):**
Turn on Sharpe, drawdown, turnover penalties:
$$L_{\text{total}}$$
 3. **Stage C (Optional RL-style Fine-Tuning):**
Use backtester rollouts and optimize a reward aligned to NP/DD & Sharpe.
-